

Customer Profiling Through Transaction Behavior for Marketing Strategy Optimization: An Unsupervised Learning Approach Using EDA, K-Means, and PCA

Hannah R. Bergman

WGU Email: hbergm3@wgu.edu

A. Research Question.....	2
Research Question.....	2
Justification.....	2
Context & Methodology.....	2
Hypothesis.....	2
B. Data Collection.....	3
Data Collected.....	3
Data Dictionary.....	3
Advantage.....	4
Disadvantage.....	4
Challenges Encountered During Data Collection.....	4
C. Data Extraction & Preparation.....	4
Tools & Technologies Used.....	4
Data Extraction & Preparation Process.....	5
Justification.....	10
Advantage.....	11
Disadvantage.....	11
D. Analysis.....	11
Analysis Techniques.....	11
Exploratory Data Analysis (EDA).....	11
K-Means Clustering.....	15
PCA for Cluster Visualization.....	18
Analysis Technique Justification.....	19
Analysis Technique Advantages.....	20
Analysis Technique Disadvantages.....	20
E. Data Summary & Implications.....	20
Results.....	20
Limitations of the Analysis.....	21
Recommended Course of Action.....	21
Directions for Future Study of the Data.....	22

F. Sources.....	22
Data & Code Sources.....	22
References.....	22

A. Research Question

Research Question

How can credit card users be segmented into distinct behavioral profiles using unsupervised learning techniques, including Exploratory Data Analysis (EDA), K-Means clustering, and Principal Component Analysis (PCA), and how effective are these segments in reflecting meaningful differences in customer purchasing and payment behavior as measured by the silhouette score?

Justification

In the highly competitive financial services industry, personalized marketing is crucial. However, many institutions still rely on broad, non-specific campaigns that don't account for the unique behavioral nuances of their customer base. This research is justified by the need to gain a deeper, data-driven understanding of diverse customer behaviors. By using unsupervised learning to cluster customers into behavioral profiles based on their transaction data, our business can tailor marketing strategies to the specific needs of each segment, ultimately improving targeting efficiency, enhancing customer retention, and driving overall profitability.

Context & Methodology

The analysis uses a dataset of 8,950 credit card users over a six-month period, which includes 18 numerical features such as balance usage, cash advances, purchase types, payment patterns, and credit limits. These features provide a comprehensive view of customer financial behavior but lack predefined labels or categories. This context makes unsupervised learning techniques, particularly K-Means clustering with PCA for dimensionality reduction, ideal for identifying hidden structures within the data. The resulting customer segments can then be used to inform strategic decisions regarding credit offerings, promotions, and risk management.

Hypothesis

Null Hypothesis (H_0): The clustering model does not produce well-separated, meaningful customer segments. The silhouette score will be significantly below 0.5, indicating poor model performance and significant overlap between clusters.

Alternative Hypothesis (H_1): The clustering model produces well-separated, meaningful customer segments. The silhouette score will be ≥ 0.5 , suggesting strong clustering structure and customer profile differentiation.

B. Data Collection

Data Collected

The "CC GENERAL" dataset on Kaggle, provided by Arjun Bhasin under a CC0: Public Domain license (Bhasin, 2018), serves as the data source for this unsupervised clustering model. The license permits unrestricted use, making the data ideal for academic projects. The dataset, provided in CSV format, contains anonymized credit card usage data for 8,950 customers over a six-month period, with 18 behavioral features that include account balances, purchase types, cash advances, payments, and credit limits. With a low rate of missing values (approximately 3.5% across two columns), the dataset is notably dense and well-suited for modeling without needing extensive imputation.

Data Dictionary

Variable	Data Type	Description
CUST_ID	Qualitative	Unique identifier for each credit card holder.
BALANCE	Quantitative	The balance amount remaining in the account.
BALANCE_FREQUENCY	Quantitative	A score (0-1) indicating how often the balance is updated.
PURCHASES	Quantitative	Total purchase amount made by the customer.
ONEOFF_PURCHASES	Quantitative	The total amount of one-time, large purchases.
INSTALLMENTS_PURCHASES	Quantitative	The total amount of installment-based purchases.
CASH_ADVANCE	Quantitative	The total cash advance amount taken.
PURCHASES_FREQUENCY	Quantitative	A score (0-1) representing the frequency of purchases.
ONEOFF_PURCHASES_FREQUENCY	Quantitative	A score (0-1) representing the frequency of one-off purchases.
PURCHASES_INSTALLMENTS_FREQUENCY	Quantitative	A score (0-1) representing the frequency of installment purchases.
CASH_ADVANCE_FREQUENCY	Quantitative	A score (0-1) representing the frequency of cash advances.
CASH_ADVANCE_TRX	Quantitative	The number of cash advance transactions.
PURCHASES_TRX	Quantitative	The number of purchase transactions.
CREDIT_LIMIT	Quantitative	The credit card limit assigned to the customer.
PAYMENTS	Quantitative	The total payment amount made by the customer.
MINIMUM_PAYMENTS	Quantitative	The total minimum payments made by the customer.

PRC_FULL_PAYMENT	Quantitative	The percentage of full payments made.
TENURE	Quantitative	The number of months the customer has held the card.

Advantage

The main advantage of the dataset is its convenience and accessibility. The public availability and permissive CC0 license make it legally unrestricted for any type of use. It's also a well-structured and clean dataset with a low rate of missing values, which significantly reduces the time and effort required for data cleaning.

Disadvantage

A primary limitation of this dataset is its lack of contextual depth due to its anonymized and static nature. It's rich in behavioral features like purchases and payments, but it lacks important time-based (e.g., seasonality), demographic (e.g., age, income, location), and qualitative data (e.g., customer feedback). This absence of context does prevent the analysis from identifying time-sensitive trends or segmenting customers based on factors like life stage or income level, which can lead to less nuanced and less actionable customer segments.

Challenges Encountered During Data Collection

The main challenges encountered involved data quality and the unsupervised nature of the task. Missing values, specifically in MINIMUM_PAYMENTS and CREDIT_LIMIT, were addressed using median imputation to maintain the integrity of feature distributions. To prepare the financial features for clustering, StandardScaler() was applied, normalizing the varying scales of attributes like balances and transaction frequencies. Outliers were identified through boxplots. Rather than being removed, they were retained and interpreted within the business context to ensure the clustering model captured the full spectrum of behavioral extremes. Finally, given the absence of pre-defined labels, K-Means clustering was used alongside Principal Component Analysis (PCA) to effectively identify and explore natural groupings within the dataset.

C. Data Extraction & Preparation

Tools & Technologies Used

Python: Python was the programming language I chose for my analysis as it's open-source (free-to-use) and offers an extensive collection of powerful libraries and packages specifically designed to make working with data and performing data science tasks easier. An advantage of python is that it has a vast array of specialized libraries that help streamline data manipulation and analysis. A disadvantage of this language is that it's generally considered "slow" as it executes code line-by-line (Gupta, 2025).

Jupyter Notebook: Using Python within Jupyter Notebook allowed for computations and visualizations to be executed in a convenient, incredibly intuitive and accessible web-based environment that requires minimal setup and maintenance. The benefit to using Jupyter Notebook as my computing environment is that code cells can be executed individually, allowing for immediate feedback and iterative development - which is ideal for data exploration, visualization and experimentation. The disadvantage to Jupyter Notebook is that the code cell benefit can also easily lead to reproducibility issues. Cells must be carefully executed in order to avoid problems with consistency, which can lead to slower processing times. For example, if you need to go back through your notebook to correct a mistake, you likely need to pause and restart the kernel again (you can't simply continue running the kernel without the possibility of causing problems).

The following Python libraries served as the core "tools" for our data extraction and preparation:

 **Pandas (pd):** Provided high-performance, user-friendly data structures (DataFrames) that mimic spreadsheets for easy, efficient data manipulation, cleaning and analysis.

 **NumPy (np):** Allowed us to more efficiently perform more complex mathematical computations across our dataset.

 **Matplotlib (plt):** Allowed for the creation of various plot types, including line graphs, boxplots, and scatter plots, crucial for data visualization and model assessment.

 **Seaborn (sns):** Building on Matplotlib, Seaborn helped produce more statistically informed and aesthetically pleasing visualizations, enhancing our exploratory data analysis.

 **Scikit-learn (sklearn):** A powerful machine learning library, Scikit-learn provided straightforward and efficient implementations of key algorithms and preprocessing tools essential to our analysis. Specific components used included:

- **SimpleImputer:** Handled missing (NaN) values. In this analysis it was used to fill in missing data points with either 0.0 (for inactive customers) or the column's median value (to preserve data distribution).
- **preprocessing (StandardScaler):** Standardized the dataset by transforming features to have a mean of 0 and a standard deviation of 1, which is crucial for distance-based algorithms like K-Means.
- **KMeans:** Implemented the K-Means clustering algorithm, which was central to our analysis for dividing data into k groups (clusters) based on feature similarity.
- **silhouette_score:** Calculated the silhouette score, a metric used to evaluate how well each data point fits into its assigned cluster relative to other clusters, assessing the quality of our clustering model.
- **PCA:** Reduced the dimensionality of the dataset by creating new composite features (principal components) that capture the most variance (information) in the data. This allowed for effective 2D visualization of cluster separation.

Data Extraction & Preparation Process

The data extraction and preparation process followed a systematic approach, moving from raw data import through cleaning, exploration, and final preprocessing.

Data Import & Setup

The initial phase involved loading the dataset and setting it up for analysis.

Data Import:

Using **Pandas**, the “CC General” dataset was read into a DataFrame

```
# Utilize the Pandas library to create a dataframe (df) to visualize our data at a high level
df = pd.read_csv('CC GENERAL.csv')
```

Initial Data Preview & Cleaning:

Pandas was used to configure display options, ensuring all columns were visible when previewing the DataFrame, and to display the first 5 rows.

```
# Preview the dataframe (displaying all columns)|
pd.set_option("display.max_columns", None)
df.head()
```

	CUST_ID	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY
0	C10001	40.900749	0.818182	95.40	0.00	95.4	0.000000	0.0
1	C10002	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	0.0
2	C10003	2495.148862	1.000000	773.17	773.17	0.0	0.000000	1.0
3	C10004	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	0.0
4	C10005	817.714335	1.000000	16.00	16.00	0.0	0.000000	0.0

Column data types and counts of missing values were inspected using **Pandas**, providing an initial overview of data quality.

```
# Review dataset column data types, value counts and total rows/columns
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8950 entries, 0 to 8949
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CUST_ID                               8950 non-null   object
1   BALANCE                               8950 non-null   float64
2   BALANCE_FREQUENCY                     8950 non-null   float64
3   PURCHASES                             8950 non-null   float64
4   ONEOFF_PURCHASES                     8950 non-null   float64
5   INSTALLMENTS_PURCHASES               8950 non-null   float64
6   CASH_ADVANCE                         8950 non-null   float64
7   PURCHASES_FREQUENCY                 8950 non-null   float64
8   ONEOFF_PURCHASES_FREQUENCY          8950 non-null   float64
9   PURCHASES_INSTALLMENTS_FREQUENCY    8950 non-null   float64
10  CASH_ADVANCE_FREQUENCY              8950 non-null   float64
11  CASH_ADVANCE_TRX                    8950 non-null   int64
12  PURCHASES_TRX                      8950 non-null   int64
13  CREDIT_LIMIT                        8949 non-null   float64
14  PAYMENTS                           8950 non-null   float64
15  MINIMUM_PAYMENTS                    8637 non-null   float64
16  PRC_FULL_PAYMENT                    8950 non-null   float64
17  TENURE                             8950 non-null   int64
dtypes: float64(14), int64(3), object(1)
memory usage: 1.2+ MB
```

A non-informative ID column (CUST_ID) was identified and dropped using **Pandas**, simplifying the dataset.

```
# Drop customer_id column since its just a unique identifier
df = df.drop(columns=["CUST_ID"])
```

Pandas was then used to check for missing values across all remaining columns, showing the extent of data incompleteness.

```
# Check for missing values in our dataset
df.isnull().sum().sort_values(ascending=False)
```

```
MINIMUM_PAYMENTS      313
CREDIT_LIMIT           1
BALANCE                0
CASH_ADVANCE_FREQUENCY 0
PRC_FULL_PAYMENT       0
PAYMENTS              0
PURCHASES_TRX          0
CASH_ADVANCE_TRX       0
PURCHASES_INSTALLMENTS_FREQUENCY 0
BALANCE_FREQUENCY      0
ONEOFF_PURCHASES_FREQUENCY 0
PURCHASES_FREQUENCY    0
CASH_ADVANCE           0
INSTALLMENTS_PURCHASES 0
ONEOFF_PURCHASES       0
PURCHASES              0
TENURE                 0
dtype: int64
```

Rows where **MINIMUM_PAYMENTS** was missing were specifically filtered using **Pandas**.

```
# Extract only the rows where MINIMUM_PAYMENTS is missing
missing_rows = df[df["MINIMUM_PAYMENTS"].isna()]
missing_rows
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	0.083333
45	2242.311686	1.000000	437.00	97.00	340.0	184.648692	0.333333
47	3910.111237	1.000000	0.00	0.00	0.0	1980.873201	0.000000
54	6.660517	0.636364	310.00	0.00	310.0	0.000000	0.666667
55	1311.995984	1.000000	1283.90	1283.90	0.0	0.000000	0.250000
...	...	...	...	...	...	...	...
8919	14.524779	0.333333	152.00	152.00	0.0	0.000000	0.333333
8929	371.527312	0.333333	0.00	0.00	0.0	1465.407927	0.000000
8935	183.817004	1.000000	465.90	0.00	465.9	0.000000	1.000000
8944	193.571722	0.833333	1012.73	1012.73	0.0	0.000000	0.333333
8946	19.183215	1.000000	300.00	0.00	300.0	0.000000	1.000000

313 rows x 17 columns

Handling Missing Values (Imputation):

Pandas provided summary statistics for selected columns in the filtered dataset, which helped to better understand the distribution of `MINIMUM_PAYMENTS`.

```
# View columns that might explain whether missing MINIMUM_PAYMENTS = 0
display(missing_rows[["BALANCE", "PURCHASES", "PAYMENTS", "CREDIT_LIMIT", "PRC_FULL_PAYMENT"]].describe())
```

	BALANCE	PURCHASES	PAYMENTS	CREDIT_LIMIT	PRC_FULL_PAYMENT
count	313.000000	313.000000	313.000000	313.000000	313.0
mean	555.441321	393.087284	322.286168	3731.789137	0.0
std	1292.687887	757.905701	1996.658905	2924.606153	0.0
min	0.000000	0.000000	0.000000	500.000000	0.0
25%	0.187069	1.400000	0.000000	1500.000000	0.0
50%	16.848358	130.400000	0.000000	3000.000000	0.0
75%	286.686616	399.950000	0.000000	5000.000000	0.0
max	9164.724752	7597.090000	29272.486070	19500.000000	0.0

Pandas, used in conjunction with **NumPy**, was used to identify and filter for likely true-zero (inactive, no minimum due) payment cases.

```
# How many missing minimums also had 0 payments and 0 purchases (no activity, no minimum due)
zero_like = missing_rows[
    (missing_rows["PAYMENTS"] == 0) &
    (missing_rows["PURCHASES"] == 0)
]
```

Missing `MINIMUM_PAYMENTS` were then imputed with 0.0 where applicable, using **Pandas**.

```
df.loc[zero_like_condition, "MINIMUM_PAYMENTS"] = 0.0
```

Remaining missing `MINIMUM_PAYMENTS` were imputed with the column's median value using **Pandas** to preserve the distribution of the data.

```
df.loc[remaining_na_condition, "MINIMUM_PAYMENTS"] = median_value
```

Missing `CREDIT_LIMIT` values were also imputed with their median using **Pandas**.

```
df["CREDIT_LIMIT"].fillna(credit_limit_median, inplace=True)
```

Exploratory Data Analysis (EDA)

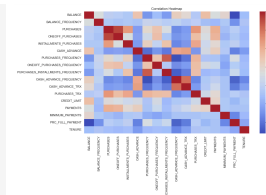
EDA was then performed to understand the dataset's structure, relationships between variables, and to detect any outliers.

Correlation Heatmap:

A figure canvas was prepared using **Matplotlib**.

Seaborn was then used to visualize the correlation heatmap of the dataset, illustrating the linear relationships between all numerical features. This helped to highlight highly correlated features.

```
# Correlation matrix heatmap
plt.figure(figsize=(14, 10))
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```



Summary Statistics:

Pandas describe() method generated comprehensive summary statistics for all numerical columns, providing key insights into the range, mean, and distribution of each variable.

```
# Summary statistics to detect obvious outliers
display(df.describe())
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENC
count	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000
mean	1564.474828	0.877271	1003.204834	592.437371	411.067645	978.871112	0.49035
std	2081.531879	0.236904	2136.634782	1659.887917	904.338115	2097.163877	0.40137
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
25%	128.281915	0.888889	39.635000	0.000000	0.000000	0.000000	0.08333
50%	873.385231	1.000000	361.280000	38.000000	89.000000	0.000000	0.50000
75%	2054.140036	1.000000	1110.130000	577.405000	468.637500	1113.821139	0.91666
max	19043.138560	1.000000	49039.570000	40761.250000	22500.000000	47137.211760	1.00000

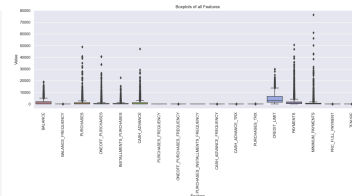
Outlier Detection (using Boxplots):

The DataFrame was reshaped (melted) using **Pandas** melt() function to more efficiently plot multiple boxplots.

```
# Boxplots to visualize outliers
df_melted = df.melt(var_name="Feature", value_name="Value")
```

A combined boxplot was generated using **Matplotlib** and **Seaborn** to detect outliers across various features in a single visualization, for quick reference.

```
plt.figure(figsize=(14, 8))
sns.boxplot(data=df_melted, x="Feature", y="Value")
plt.xticks(rotation=90)
plt.title("Boxplots of all Features")
plt.tight_layout()
plt.show()
```



Individual subplots were generated using **Matplotlib** to provide more detailed individual feature boxplots for features that showed obvious outliers in the previous step.

Seaborn rendered these individual boxplots, allowing us to more closely examine outlier presence and distribution for each feature.

```
# Zoomed boxplots of noticeable outlier-prone features
sns.set_style("whitegrid")
sns.color_palette("Set2")
fig, ax = plt.subplots(4, 2, figsize=(12, 10))

# BALANCE
sns.boxplot(x=df['BALANCE'], ax=ax[0,0], color='cornflowerblue')
ax[0,0].set_title('BALANCE')
ax[0,0].set_xlabel('Balance Left in Account (USD)')

# PURCHASES
sns.boxplot(x=df['PURCHASES'], ax=ax[0,1], color='cornflowerblue')
ax[0,1].set_title('PURCHASES')
ax[0,1].set_xlabel('Amount of Purchases Made From Account (USD)')

# ONEOFF_PURCHASES
sns.boxplot(x=df['ONEOFF_PURCHASES'], ax=ax[1,0], color='cornflowerblue')
ax[1,0].set_title('ONEOFF_PURCHASES')
ax[1,0].set_xlabel('Max Purchase Amount Done in One-Go (USD)')

# INSTALLMENTS_PURCHASES
sns.boxplot(x=df['INSTALLMENTS_PURCHASES'], ax=ax[1,1], color='cornflowerblue')
ax[1,1].set_title('INSTALLMENTS_PURCHASES')
ax[1,1].set_xlabel('Amount of Purchase Done in Installment (USD)')

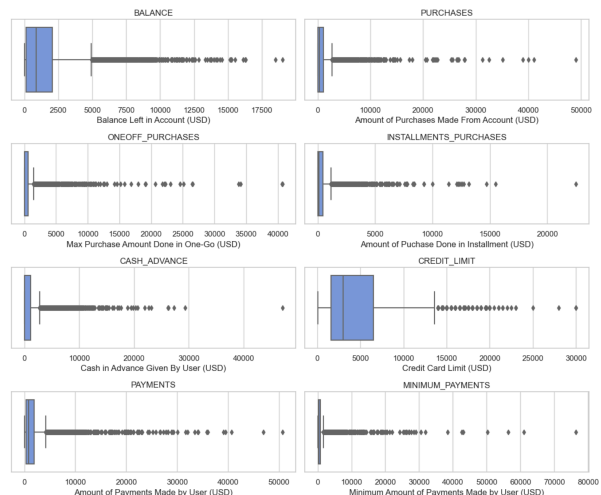
# CASH_ADVANCE
sns.boxplot(x=df['CASH_ADVANCE'], ax=ax[2,0], color='cornflowerblue')
ax[2,0].set_title('CASH_ADVANCE')
ax[2,0].set_xlabel('Cash in Advance Given By User (USD)')

# CREDIT_LIMIT
sns.boxplot(x=df['CREDIT_LIMIT'], ax=ax[2,1], color='cornflowerblue')
ax[2,1].set_title('CREDIT_LIMIT')
ax[2,1].set_xlabel('Credit Card Limit (USD)')

# PAYMENTS
sns.boxplot(x=df['PAYMENTS'], ax=ax[3,0], color='cornflowerblue')
ax[3,0].set_title('PAYMENTS')
ax[3,0].set_xlabel('Amount of Payments Made by User (USD)')

# MINIMUM_PAYMENTS
sns.boxplot(x=df['MINIMUM_PAYMENTS'], ax=ax[3,1], color='cornflowerblue')
ax[3,1].set_title('MINIMUM_PAYMENTS')
ax[3,1].set_xlabel('Minimum Amount of Payments Made by User (USD)')

plt.tight_layout()
plt.show()
```



 **Pandas** was used to retrieve the top 10 outlier values for each feature...

```
df[feature].sort_values(ascending=False).head(10).reset_index(drop=True)
```

Compiling them into a new table for further review.

```
top_outliers_df = pd.DataFrame(top_outliers)
display(top_outliers_df)
```

	BALANCE	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	CREDIT_LIMIT	PAYMENTS	MINIMUM_PAYMENTS
0	19043.13856	49039.57	40761.25	22500.00	47137.21176	30000.0	50721.48336	76406.20752
1	18495.55855	41050.40	40624.06	15497.19	29282.10915	30000.0	46930.59824	61031.61860
2	16304.88925	40040.71	34087.73	14686.10	27296.48576	28000.0	40627.59524	56370.04117
3	16259.44857	38902.71	33803.84	13184.43	26268.69989	25000.0	39461.96580	50260.75947
4	16115.59640	35131.16	26547.43	12738.47	26194.04954	23000.0	39048.59762	43132.72823
5	15532.33972	32539.78	26514.32	12560.85	23130.82106	23000.0	36066.75068	42629.55117
6	15258.22590	31299.35	25122.77	12541.00	22665.77850	22500.0	35843.62593	38512.12477
7	15244.74865	27957.68	24543.52	12375.00	21943.84942	22000.0	34107.07499	31871.36379
8	15155.53286	27790.42	23032.97	12235.05	20712.67008	21500.0	33994.72785	30528.43240
9	14581.45914	26784.62	22257.39	12128.94	20277.33112	21500.0	33486.31044	29019.80288

Data Preprocessing

This stage involved transforming the data even further after our preliminary cleaning and exploration, to ensure it would be suitable for our K-Means algorithm.

Rounding Numeric Values:

 All numeric values in the DataFrame were rounded down to 2 decimal places using **Pandas**, ensuring data consistency and precision for subsequent steps.

```
# Round to 2 decimal places
df = df.round(2)
```

Feature Standardization:

 **Scikit-learn's** StandardScaler was applied to standardize the features. This step transforms our data so that it has a mean of 0 and a standard deviation of 1. Standardization is essential for distance-based algorithms like K-Means, as it prevents features with larger scales from disproportionately influencing the clustering results (Habib, 2024).

```
# Initialize the scaler
scaler = StandardScaler()

# Fit the scaler to the data and transform it
scaled_data = scaler.fit_transform(df)
```

Saving Cleaned Data:

 The transformed dataset was saved to a CSV file using **Pandas**, creating a clean snapshot of the data ready for modeling.

```
# Save the transformed dataset as a CSV file
df.to_csv('d214_task2_cc_clean.csv', index=False)
```

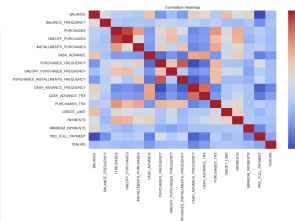
Justification

Advantage

Disadvantage

D. Analysis

Analysis Techniques

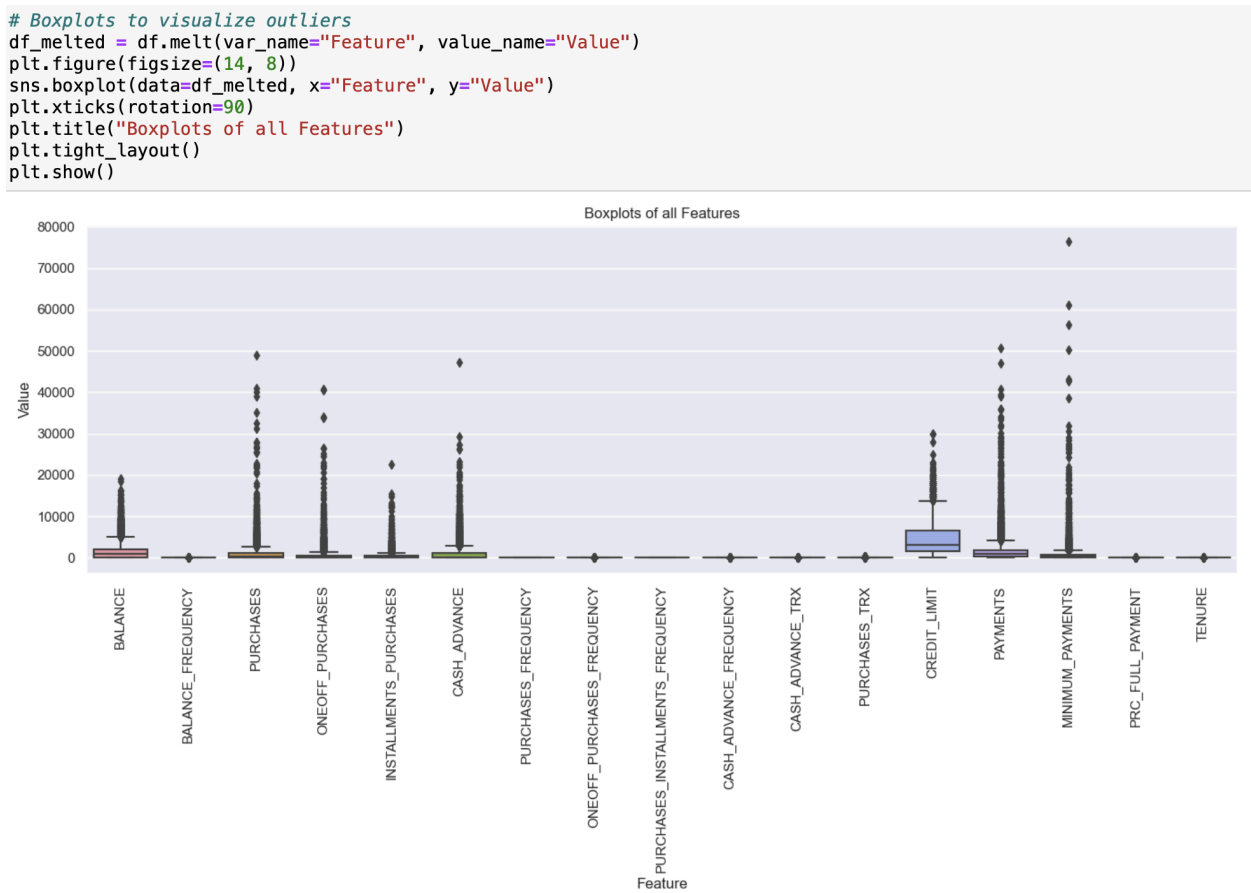


Descriptive Statistics: The describe() method from Pandas was used to produce summary statistics for all numerical columns, providing key insights into the range, mean, and distribution of each variable.

```
# Summary statistics to detect obvious outliers
display(df.describe())
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY
count	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000
mean	1564.474828	0.877271	1003.204834	592.437371	411.067645	978.871112	0.49035
std	2081.531879	0.236904	2136.634782	1659.887917	904.338115	2097.163877	0.40137
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
25%	128.281915	0.888889	39.635000	0.000000	0.000000	0.000000	0.08333
50%	873.385231	1.000000	361.280000	38.000000	89.000000	0.000000	0.50000
75%	2054.140036	1.000000	1110.130000	577.405000	468.637500	1113.821139	0.91666
max	19043.138560	1.000000	49039.570000	40761.250000	22500.000000	47137.211760	1.00000

Outlier Detection: Boxplots were created to quickly identify outliers across all features.



More detailed individual boxplots were then generated for features with noticeable significant outliers to enable a closer examination of their distribution.

```

# Zoomed boxplots of noticeable outlier-prone features
sns.set_style("whitegrid")
sns.color_palette("Set2")
fig, ax = plt.subplots(4, 2, figsize=(12, 10))

# BALANCE
sns.boxplot(x=df['BALANCE'], ax=ax[0,0], color='cornflowerblue')
ax[0,0].set_title('BALANCE')
ax[0,0].set_xlabel('Balance Left in Account (USD)')

# PURCHASES
sns.boxplot(x=df['PURCHASES'], ax=ax[0,1], color='cornflowerblue')
ax[0,1].set_title('PURCHASES')
ax[0,1].set_xlabel('Amount of Purchases Made From Account (USD)')

# ONEOFF_PURCHASES
sns.boxplot(x=df['ONEOFF_PURCHASES'], ax=ax[1, 0], color='cornflowerblue')
ax[1,0].set_title('ONEOFF_PURCHASES')
ax[1,0].set_xlabel('Max Purchase Amount Done in One-Go (USD)')

# INSTALLMENTS_PURCHASES
sns.boxplot(x=df['INSTALLMENTS_PURCHASES'], ax=ax[1, 1], color='cornflowerblue')
ax[1,1].set_title('INSTALLMENTS_PURCHASES')
ax[1,1].set_xlabel('Amount of Purchase Done in Installment (USD)')

# CASH_ADVANCE
sns.boxplot(x=df['CASH_ADVANCE'], ax=ax[2, 0], color='cornflowerblue')
ax[2,0].set_title('CASH_ADVANCE')
ax[2,0].set_xlabel('Cash in Advance Given By User (USD)')

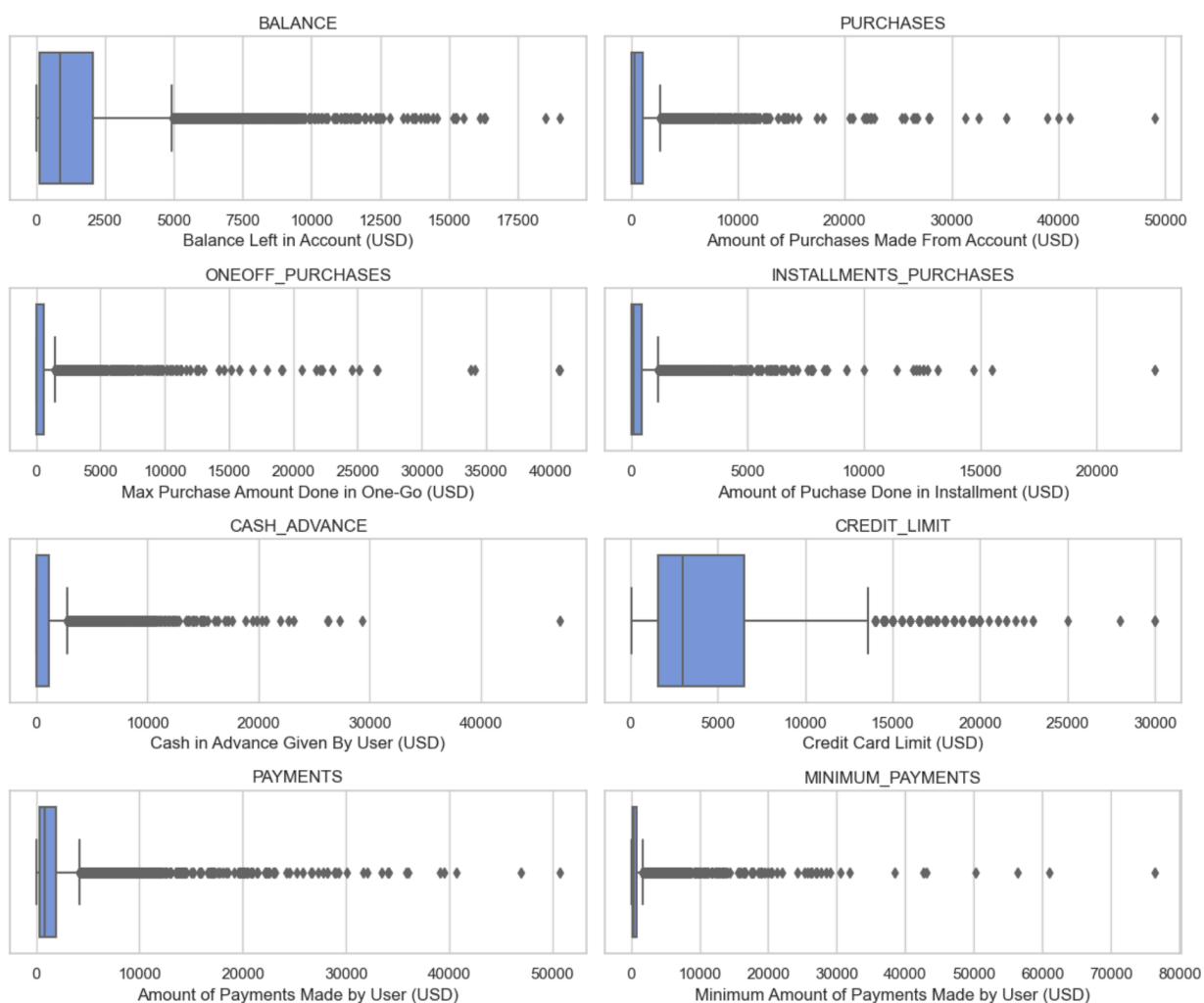
# CREDIT_LIMIT
sns.boxplot(x=df['CREDIT_LIMIT'], ax=ax[2, 1], color='cornflowerblue')
ax[2,1].set_title('CREDIT_LIMIT')
ax[2,1].set_xlabel('Credit Card Limit (USD)')

# PAYMENTS
sns.boxplot(x=df['PAYMENTS'], ax=ax[3, 0], color='cornflowerblue')
ax[3,0].set_title('PAYMENTS')
ax[3,0].set_xlabel('Amount of Payments Made by User (USD)')

# MINIMUM_PAYMENTS
sns.boxplot(x=df['MINIMUM_PAYMENTS'], ax=ax[3, 1], color='cornflowerblue')
ax[3,1].set_title('MINIMUM_PAYMENTS')
ax[3,1].set_xlabel('Minimum Amount of Payments Made by User (USD)')

plt.tight_layout()
plt.show()

```



Pandas was used to compile the top 10 outlier values for each feature into a separate table for further review.

```
# Inspect top 10 outlier values by feature
features_to_inspect = ["BALANCE", "PURCHASES", "ONEOFF_PURCHASES", "INSTALLMENTS_PURCHASES", "CASH_ADVANCE", "CREDIT_LIMIT", "PAYMENTS", "MINIMUM_PAYMENTS"]
top_outliers = {}

for feature in features_to_inspect:
    top_outliers[feature] = df[feature].sort_values(ascending=False).head(10).reset_index(drop=True)

top_outliers_df = pd.DataFrame(top_outliers)
display(top_outliers_df)
```

	BALANCE	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	CREDIT_LIMIT	PAYMENTS	MINIMUM_PAYMENTS
0	19043.13856	49039.57	40761.25	22500.00	47137.21176	30000.0	50721.48336	76406.20752
1	18495.55855	41050.40	40624.06	15497.19	29282.10915	30000.0	46930.59824	61031.61860
2	16304.88925	40040.71	34087.73	14686.10	27296.48576	28000.0	40627.59524	56370.04117
3	16259.44857	38902.71	33803.84	13184.43	26268.69989	25000.0	39461.96580	50260.75947
4	16115.59640	35131.16	26547.43	12738.47	26194.04954	23000.0	39048.59762	43132.72823
5	15532.33972	32539.78	26514.32	12560.85	23130.82106	23000.0	36066.75068	42629.55117
6	15258.22590	31299.35	25122.77	12541.00	22665.77850	22500.0	35843.62593	38512.12477
7	15244.74865	27957.68	24543.52	12375.00	21943.84942	22000.0	34107.07499	31871.36379
8	15155.53286	27790.42	23032.97	12235.05	20712.67008	21500.0	33994.72785	30528.43240
9	14581.45914	26784.62	22257.39	12128.94	20277.33112	21500.0	33486.31044	29019.80288

K-Means Clustering

With the data prepared, the core clustering algorithm was applied to segment the credit card users.

Finding the Optimal Number of Clusters (K): To determine the optimal number of clusters, I opted to use the Elbow Method, a popular technique used to systematically determine the best value in K-Means clustering (GeeksforGeeks, 2025). This method plots the number of clusters (k) against the inertia (the sum of squared distances of samples to their closest cluster center). The goal is to find the "elbow" point in the curve where the rate of decrease in inertia slows down significantly, indicating that adding more clusters provides diminishing returns (Saji, 2025).

To start, an empty list named inertia was created to store the inertia value for each k.

```
# Create an empty list to store the inertia value for each value of k
inertia = []
```

Next, a range of k values from 1 to 10 was defined to test, fitting a K-Means model for each of these values to see how the performance (inertia) changes.

```
# Define a range of cluster numbers (k) to test
k_range = range(1, 10)
```

A for loop was used to iterate through each k. For each value, a K-Means model from Scikit-learn was instantiated with n_init=10, allowing the algorithm to run 10 times with different centroid seeds and pick the best result (Scikit-learn, 2025), providing more stable and accurate clustering.

```
for k in k_range:
    # Create a KMeans model for the current value of k
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
```

The KMeans model was then fitted on the scaled dataset (scaled_data), creating k clusters. During this step, the algorithm assigned data points to clusters and calculated the centroid

positions.

```
# Fit the Kmeans model on the scaled dataset
kmeans.fit(scaled_data)
```

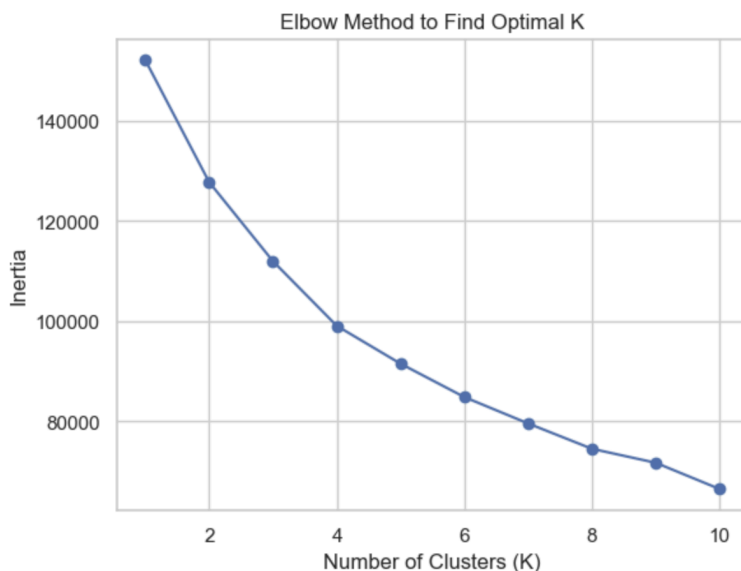
The inertia value from the fitted model was then added to the inertia list. This value reflects how tight and compact clusters are for that k. Lower inertia is better, but adding more clusters typically lowers inertia, in general. With this, we're specifically looking for a point where it stops improving sharply (the "elbow").

```
# Add inertia value from the fitted model to the inertia list
inertia.append(kmeans.inertia_)
```

Finally, we plotted a line chart of the k values vs. inertia to visualize the results, looking for the "elbow" point in the curve where the drop in inertia slows down, where the corresponding k value will be the optimal number of clusters for our data.

```
# Plot elbow curve - a line chart of K values vs. inertia
plt.plot(k_range, inertia, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method to Find Optimal K')
plt.show()
```

Interpreting our chart, we can see that the "elbow" appears around K=3. This is where the biggest drops taper off and clustering becomes less cost-effective beyond that so, therefore, the optimal number of clusters is 3.



Based on the Elbow Method, the optimal number of clusters was set to K=3. A final K-Means model was created with these parameters and fitted to the standardized data.

```
# Define optimal k
optimal_k = 3
```

We initialized a K-Means clustering model utilizing scikit-learn's KMeans algorithm, setting the number of clusters (K=3) and number of times the K-Means algorithm will run (n_init=10 - this will run the KMeans algorithm 10 times with different starting centroids, picking the best result,

the one with the lowest inertia).

```
# Initialize KMeans model
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
```

Cluster Labeling: We fit the KMeans model to the standardized dataset (scaled_data) and the algorithm will assign each data point to the closest cluster centroid, predicting which cluster each data point belongs to. The result, stored as 'clusters', is a list of cluster labels (0, 1, or 2) assigned to each customer.

```
# Fit Kmeans model to standardized dataset
clusters = kmeans.fit_predict(scaled_data)
```

A new column called 'Cluster' is added to our DataFrame (df) so each row (customer) includes their cluster assignment. This allows us to easily analyze and visualize the clusters.

```
# Add cluster labels to the original dataset
df['Cluster'] = clusters
```

Using the newly created 'Cluster' column, we can group our dataset, where all rows belonging to the same cluster are grouped together. For each cluster, we calculate the average (mean) of the other columns, resulting in a summary table showing the average profile for each cluster.

```
# Cluster summary table
cluster_summary = df.groupby('Cluster').mean().round(2)

print("Cluster Summary Statistics:")
display(cluster_summary)
```

Cluster Summary Statistics:

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY
Cluster							
0	2185.55	0.98	4191.03	2668.17	1523.32	450.46	0.95
1	803.84	0.83	497.25	247.50	250.07	336.25	0.46
2	4005.36	0.96	386.99	251.10	135.97	3891.14	0.23

Cluster Profiling: The clusters were profiled by calculating the average value of each feature for each cluster. This process revealed three distinct behavioral profiles, which were given descriptive labels:

	Cluster 0:	Cluster 1:	Cluster 2:
Feature	Value	Value	Value
PURCHASES	\$4,191.03	\$497.03	\$386.99
BALANCE	\$2,185.55	\$803.84	\$4,005.36
CREDIT_LIMIT	\$7,650.55	\$3,265.45	\$6,699.69
PURCHASES_FREQUENCY	0.95	0.46	0.23
ONEOFF_PURCHASES_FREQUENCY	0.67	0.13	0.11
PURCHASES_INSTALLMENTS_FREQUENCY	0.74	0.34	0.15
CASH_ADVANCE_FREQUENCY	0.06	0.07	0.45
CASH_ADVANCE_TRX	1.51	1.22	12.49
PRC_FULL_PAYMENT	0.30	0.15	0.03

PAYMENTS	\$4,079.47	\$908.03	\$3,030.76
MINIMUM_PAYMENTS	\$1,227.53	\$526.96	\$1,751.82

❖ **Cluster 0: Engaged High Spenders with Responsible Credit Use**

These customers demonstrate high purchase volumes, frequent transactions, and responsible credit behavior, including a high percentage of full payments. They have the highest credit limits and use their cards for both one-off and installment purchases. They are likely a profitable and low-risk segment.

❖ **Cluster 1: Low Engagement, Cautious Users**

This group has the lowest spending and credit limit. They are infrequent card users, showing minimal activity across all features. Their behavior suggests they may be new to credit or use their cards only for occasional, small purchases.

❖ **Cluster 2: Cash Advance Reliant Revolvers**

This segment is characterized by a heavy reliance on cash advances, with the highest cash advance amount, frequency, and number of transactions. Despite having high balances, their purchase activity is low, and their repayment behavior is poor (the lowest percentage of full payments). This group likely faces financial distress and represents a higher-risk segment for credit providers.

Cluster Separation Evaluation: Cluster quality was evaluated using the silhouette score, which measures the similarity of each point to its own cluster versus other clusters (Gultekin, 2023). The score ranges from -1 (very poor clustering) to +1 (excellent clustering). A score closer to +1 indicates better cluster separation, which is optimal. For effective segmentation and good clustering, a score of 0.5 or higher is required based on our research question. Scores lower than this, such as 0.2 to 0.5, indicate only moderate structure; scores even lower, below 0.2, suggest poor clustering, implying potential overlap or randomness.

Scikit-learn's `silhouette_score` function was used to evaluate how well-separated the clusters are. A higher silhouette score (closer to +1) indicates that data points are well-matched to their own cluster and poorly matched to neighboring clusters, signifying better-defined clusters. Conversely, a lower silhouette score (closer to -1) implies that data points might be in the wrong cluster or that clusters are poorly separated (Jude, 2024).

```
# Calculate silhouette score
sil_score = silhouette_score(scaled_data, clusters)
print(f"Silhouette Score: {sil_score:.4f}")
```

Silhouette Score: 0.2499

The resulting silhouette score was 0.2499. This value indicates the presence of a weak cluster structure, suggesting that while some clusters are distinct (Cluster 0), others may have significant overlap. This score aligns with the interpretation that the clusters are not perfectly separated and that some customers may fall near cluster boundaries.

PCA for Cluster Visualization

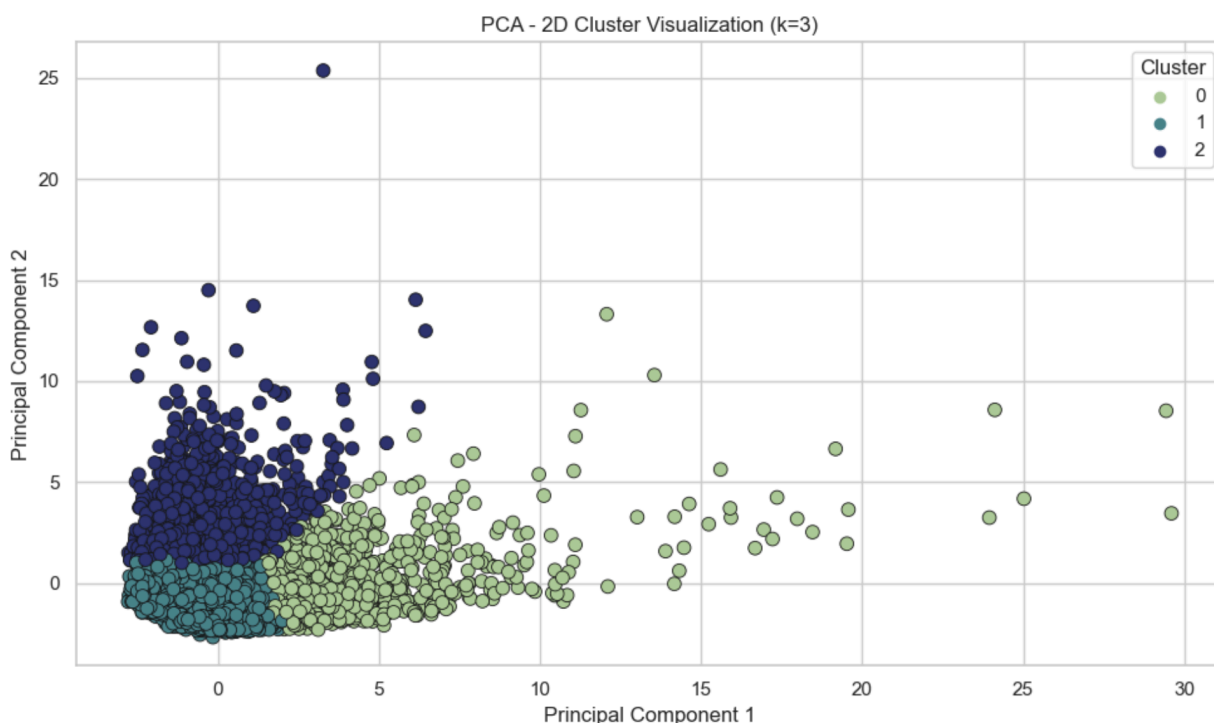
To visually confirm the findings, Principal Component Analysis (PCA) was used to reduce the data's dimensionality to two principal components. A 2D scatter plot was then generated to visualize the clusters.

Dimensionality Reduction: Scikit-learn's PCA (Principal Component Analysis) was employed to reduce the feature dimensions from many to 2, making the dataset suitable for 2D plotting. PCA transforms the original features into a new set of uncorrelated principal components, capturing the maximum variance (Chatterjee, 2025).

```
# Visualize clusters with PCA
pca = PCA(n_components=2)
principal_components = pca.fit_transform(scaled_data)
```

2D Cluster Visualization: Plotting the clusters visually confirmed the silhouette score's interpretation. The PCA scatter plot shows that Cluster 0 (Light Green) is relatively well-separated from the other two groups, indicating it represents a cohesive and distinct behavioral segment. Cluster 1 (Teal) and Cluster 2 (Dark Navy) show significant overlap, particularly along the first principal component (PC1). This overlap suggests these two clusters share similar underlying characteristics, making them less distinct. The limited separation between Clusters 1 and 2 likely contributes to the moderate silhouette score of 0.2499, reflecting that while one cluster is clearly defined, the others are less well-differentiated.

```
# Visualize clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Cluster', palette='crest', s=60, edgecolor='k')
plt.title(f"PCA - 2D Cluster Visualization (k={optimal_k})")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Cluster")
plt.tight_layout()
plt.show()
```



Analysis Technique Justification

This analysis used a combination of Exploratory Data Analysis (EDA), K-Means clustering, and Principal Component Analysis (PCA) to uncover distinct behavioral profiles among credit card

users. These three techniques form a standard and complementary workflow in unsupervised machine learning. EDA served as the foundational first step. It was used to summarize the dataset's main characteristics through visualizations and statistical summaries. This allowed for the early identification of missing values, skewed distributions, and feature correlations, which were critical for guiding data preprocessing steps like median imputation and standardization. K-Means clustering was the core modeling technique. Its purpose was to segment customers into behaviorally distinct groups. K-Means is particularly well-suited for this task due to its simplicity, scalability, and effectiveness in handling the numerical data present in this study. PCA was employed as a dimensionality reduction technique. By transforming the original, high-dimensional features into a smaller number of uncorrelated principal components, PCA allowed for the visualization of the clustering structure in two dimensions. This enhanced the interpretability of the results by making it possible to visually assess cluster separation.

Analysis Technique Advantages

The primary advantage of K-Means clustering is its computational efficiency and simplicity. Its straightforward algorithm provides a clear and intuitive way to group customers based on their behavior, making it a great tool for discovering hidden structures within large numerical datasets. This efficiency makes it a practical choice for small to large scale customer segmentation projects like ours.

Analysis Technique Disadvantages

Despite its advantages, K-Means clustering has a few key disadvantages. The algorithm assumes clusters are spherical and of equal size which, of course, may not accurately reflect the true structure of real-world behavioral data. Its performance can also be sensitive to the initial placement of centroids. Additionally, the number of clusters (k) must be predefined, which requires an additional analysis step, like the Elbow Method, to determine the optimal number of clusters and avoid a significant influence on the segmentation outcome.

E. Data Summary & Implications

Results

The research question guiding this study was:

"How can credit card users be segmented into distinct behavioral profiles using unsupervised learning techniques (EDA, K-Means, and PCA), and how effective are these segments in reflecting meaningful differences in customer purchasing and payment behavior as measured by the silhouette score?"

The results of the K-Means clustering analysis revealed three distinct customer segments with observable behavioral differences:

- **Cluster 0:** High-spending, high-engagement users who frequently pay in full, likely low-risk, high-value customers.
- **Cluster 1:** Low-activity, cautious users with minimal credit engagement - possibly new or dormant accounts.
- **Cluster 2:** Customers relying heavily on cash advances, with low spending and poor repayment behavior - potentially high-risk users.

The silhouette score for this model was 0.2499, which is considerably below the 0.5 threshold for well-separated clusters. This indicates that while the clustering revealed meaningful behavioral traits, the separation between groups is relatively weak, and there may be significant overlap in behaviors across clusters.

Limitations of the Analysis

An obvious limitation of the analysis is the low silhouette score (0.2499), which suggests that the clusters formed aren't highly cohesive or well-separated. This may be due to the assumptions of the K-Means algorithm, which expects spherical, equally sized clusters. Real-world behavioral data often violate these assumptions, especially when outliers (like heavy cash advance users) or overlapping behaviors are present. Additionally, as discussed in Section B, the analysis is based solely on static numeric features without incorporating time-based trends or contextual variables. This lack of contextual depth does prevent the analysis from identifying time-sensitive trends or segmenting customers based on factors like life stage or income level, which can lead to less nuanced and less actionable customer segments.

Recommended Course of Action

Despite the moderate effectiveness of the clustering model, a practical recommendation is to use the current segmentation as a baseline for targeted customer strategies, with refinements applied over time:

- **Cluster 0** users can be offered loyalty or cashback incentives to maintain engagement and profitability.
- **Cluster 1** users may benefit from education campaigns or credit-building tools to increase utilization.
- **Cluster 2** should be the focus of risk mitigation strategies, such as reduced access to cash advances, closer monitoring, or outreach offering financial counseling.

Even though the clusters are not highly distinct, the behavioral patterns suggest initial opportunities for differentiated engagement strategies that can be tested and iteratively improved.

Directions for Future Study of the Data

To address the limitations and deepen insights, we can attempt to apply advanced or non-linear clustering techniques, such as DBSCAN, HDBSCAN, or Gaussian Mixture Models (GMMs), to better capture irregular, overlapping, or outlier-heavy clusters that K-Means struggles to define (Sukup, 2018).

We may also want to consider integrating additional features to enhance the dataset with time-based behavior patterns (e.g., changes in spending, late payment trends, seasonal usage) or contextual variables (e.g., demographics, income tiers) to enable more dynamic segmentation that better reflects evolving user behavior and provides stronger clustering separation.

F. Sources

Data & Code Sources

Bhasin, A. (2018). *Credit Card Dataset for Clustering* [Dataset]. Kaggle.
Retrieved July 1, 2025, from
kaggle.com/datasets/arjunbhasin2013/ccdata

References

- Chatterjee, A. (April 3, 2025). *Principal Component Analysis (PCA): A Key Dimensionality Reduction Technique*. LinkedIn.
Retrieved July 23, 2025 from
[linkedin.com/pulse/principal-component-analysis-pca-key-dimensionality](https://www.linkedin.com/pulse/principal-component-analysis-pca-key-dimensionality)
- GeeksforGeeks. (July 11, 2025). *Elbow Method for Optimal Value of k in KMeans*. GeeksforGeeks.
Retrieved August 14, 2025 from
[geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k](https://www.geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k)
- Gupta, G. (July 10, 2025). *Advantages and Disadvantages of Python - Make a Favorable Decision*. Squareboat.
Retrieved July 23, 2025 from
squareboat.com/blog/advantages-and-disadvantages-of-python
- Gultekin, H. (2023, September 7). *What is a Silhouette Score?* Medium.
Retrieved March 21, 2025 from
<https://medium.com/@hazallgultekin/what-is-silhouette-score-f428fb39bf9a>
- Habib, J. (September 26, 2024). *Why and When to Standardize Data in Machine Learning: A Comprehensive Guide*. Medium.
Retrieved July 24, 2025 from
medium.com/@jaberi.mohamedhabib/why-and-when-to-standardize-your-data
- Jude, D. (December 10, 2024). *Understanding the Silhouette Coefficient in Clustering*. Medium.
Retrieved July 24, 2025 from
danieljude1992.medium.com/understanding-the-silhouette-coefficient-in-clustering
- Saji, B. (April 4, 2025). *Elbow Method for Optimal Cluster Number in K-Means*. Analytics Vidhya.

Retrieved July 24, 2025 from
analyticsvidhya.com/blog/2021/01/in-depth-intuition-of-k-means-clustering-algorithm

Scikit-learn. (2025). *KMeans*. [API Reference]. Scikit-learn.
Retrieved August 11, 2025 from
scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

Sukup, J. (February 19, 2018). *When K-Means Clustering Fails: Alternatives for Segmenting Noisy Data*. Oracle AI & Data Science Blog.
Retrieved August 14, 2025 from
blogs.oracle.com/ai-and-datascience/post/when-k-means-clustering-fails